

Trabajo Práctico Final — Entregable 2

Caso: Plataforma Magic (*MagicDB*)

Autor: Barrios, Julieta Luján

Materia: Base de Datos - 3er año Ingeniería Informática

Año: 2026

Contenido de esta entrega

Este entregable corresponde al **modelo físico** sumado a las **cargas de datos** del TP Final.

Archivo	Contenido
SQLQuery1.sql	CREATE DATABASE MagicDB. Elimina y recrea la base si ya existe, forzando el cierre de conexiones activas.
SQLQuery2.sql	Las 12 tablas con todas las restricciones nombradas explícitamente (PK_, FK_, UQ_, CK_) y política de ON DELETE definida en cada clave foránea.
SQLQuery3.sql	Inserción de datos sintéticos mediante lógica procedural T-SQL con bloques WHILE, RAND(), NEWID() y fechas relativas. Garantiza un volumen mínimo de 1000 registros en las tablas principales.
Trabajo Práctico Final - Grupo 3 - Entregable 2.pdf	Este documento: detalla los volúmenes generados, las distribuciones elegidas y el motivo de cada decisión.

1. Cómo ejecutar

```
-- 1. Abrir SSMS conectado al motor (no a una base) y ejecutar:  
-- SQLQuery1.sql  
  
-- 2. Una vez creada MagicDB, ejecutar sobre esa base:  
-- SQLQuery2.sql  
  
-- 3. Ejecutar la carga de datos sobre MagicDB:  
-- SQLQuery3.sql
```

Los tres scripts deben ejecutarse en orden. Si se vuelve a ejecutar `SQLQuery1.sql`, la base se elimina y recrea desde cero, por lo que los scripts 2 y 3 deben correrse nuevamente.

2. Herramientas utilizadas

T-SQL procedural con WHILE, RAND(), NEWID() y CHECKSUM(), ejecutado directamente en SQL Server Management Studio. Se optó por esta herramienta y no por Python + Faker por las siguientes razones:

Integración nativa: al correr todo dentro del motor, no se requieren dependencias externas ni configuración de entorno Python. Cualquier integrante del grupo puede ejecutar los tres scripts desde SSMS sin instalaciones adicionales.

Control de integridad en origen: al generar los datos dentro del mismo motor que aplica las restricciones, cualquier violación de CHECK, FK o UNIQUE falla en el momento exacto de la inserción, lo que facilita detectar errores de distribución antes de avanzar.

Uso de tablas temporales: el script utiliza `#CompradoresPermitidos` para forzar que solo un subconjunto de 24 usuarios realice compras, simulando una tasa de conversión realista. Este tipo de lógica condicional es directamente expresable en T-SQL sin necesidad de capas externas.

Distribución horaria ajustada: el script detecta la hora generada para cada evento y, con un 70% de probabilidad, desplaza los eventos generados en horario de madrugada o mañana hacia la tarde-noche, simulando el comportamiento real de las transmisiones. Esta lógica es natural en T-SQL con `DATEPART` y `DATEADD`.

3. Volúmenes generados

Tabla	Filas	Justificación
Planes	4	Catálogo fijo: <i>free</i> , <i>agency</i> , <i>pro</i> y <i>premium</i> . Representan niveles comerciales típicos de un SaaS con precios y límites diferenciados.
Usuarios	60	Base suficiente para probar relaciones, suscripciones, compras y eventos con variedad. Simula una plataforma en crecimiento temprano.
Suscripciones	79 (aprox.)	60 usuarios base, pero los usuarios divisibles por 5 tienen dos registros (upgrade) y los divisibles por 7 también (downgrade), generando historial de cambios de plan.
EstadoSuscripcion	4	Tabla referencial con los cuatro estados posibles: <i>trialing</i> , <i>active</i> , <i>past_due</i> y <i>canceled</i> .
Proyectos	Variable	Depende del plan: los usuarios <i>free</i> tienen 1 proyecto; <i>agency</i> , 1 o 2; <i>pro</i> , hasta 3; <i>premium</i> , hasta 4. Refleja que los planes superiores habilitan mayor uso del producto.
AlertasConfiguraciones	Variable	Cada proyecto recibe 2 alertas fijas (<i>follow</i> y <i>sub</i>) más alertas opcionales de tip (70% de probabilidad) y raid (40% de probabilidad), generando entre 2 y 4 configuraciones por proyecto.
TipoAlerta	5	Tabla referencial con los cinco tipos de alerta: <i>follow</i> , <i>sub</i> , <i>tip</i> , <i>raid</i> y <i>custom</i> .

CatalogoJuegos	10	Catálogo estático con 9 juegos activos y 1 inactivo. Incluye juegos gratuitos y pagos de tipo <i>html</i> y <i>download</i> .
JuegosUsuarios	Variable	Cada usuario adquiere entre 0 y 4 juegos de forma aleatoria. Permite analizar adopción del módulo de gamificación.
AssetsTienda	14	Catálogo estático con assets de tipo <i>audio</i> , <i>video</i> , <i>image</i> y <i>font</i> . Incluye un asset gratuito para mayor realismo.
ComprasAssets	1.050	Tabla principal. Supera el mínimo de 1.000 registros. Solo 24 de los 60 usuarios realizan compras, forzando una tasa de conversión del 40%.
AlertasEventos	1500	Tabla principal. Supera el mínimo de 1.000 registros. Incluye distribución de donaciones por niveles y ajuste horario para simular picos de transmisión.
Total	2.759+ (aprox.)	Sin contar proyectos y configuraciones de alertas, que varían según la lógica procedural.

4. Decisiones de distribución

4.1 Planes y distribución de usuarios

Se cargan cuatro planes con precios y límites distintos. *Free* y *agency* tienen precio 0, *pro* cuesta \$10 y *premium* cuesta \$16. Para los usuarios que reciben un plan estándar sin historial, la distribución es: 30% *free*, 30% *agency*, 25% *pro* y 15% *premium*. Esto genera una base mayoritariamente en planes gratuitos, lo que refleja el comportamiento típico de adopción de un SaaS donde los planes de pago son minoría.

Por qué importa: la pregunta de negocio 2 sobre carga operativa por plan solo tiene respuesta interpretable si los planes tienen distinta representación. Si todos tuvieran el mismo peso, el porcentaje de alertas por plan sería uniforme y el indicador no aportaría valor analítico.

4.2 Upgrades y downgrades simulados

Los usuarios cuyo `IdUsuario` es divisible por 5 comienzan en *free* cancelado y pasan a *pro* activo. Los divisibles por 7 (sin coincidir con la regla anterior) comienzan en *premium* cancelado y pasan a *agency* activo. Esto genera aproximadamente 12 upgrades y 7 downgrades sobre los 60 usuarios.

Por qué importa: la pregunta de negocio 5 sobre upgrades y downgrades requiere que existan ambos movimientos en los datos. Si todos los usuarios tuvieran una única suscripción activa, la vista `v_UpgradesDowngradesSuscripciones` con `LAG()` no detectaría ningún cambio y el indicador estaría vacío.

4.3 Compradores restringidos a un subconjunto

El script crea una tabla temporal `#CompradoresPermitidos` con 24 usuarios seleccionados aleatoriamente sobre los 60 totales. Todas las 1.050 compras se generan exclusivamente dentro de ese subconjunto. Esto fuerza una tasa de conversión de tienda del 40%.

Por qué importa: la pregunta de negocio 3 mide qué porcentaje de usuarios registrados realiza al menos una compra. Si todos los usuarios compraran, la tasa sería 100% y no habría nada que analizar. Forzar el 40% genera un indicador con valor real para la toma de decisiones comerciales.

4.4 Distribución de donaciones en eventos de tipo tip

Los eventos de alerta de tipo *tip* generan un monto donado con distribución en tres niveles: el 75% son donaciones pequeñas (entre \$1 y \$15), el 20% son donaciones medianas (entre \$15 y \$50) y el 5% son donaciones grandes (entre \$50 y \$300). Para todos los demás tipos de alerta, `ValorDonado` almacena el valor 0.00.

Por qué importa: esta distribución refleja el comportamiento real de las donaciones en plataformas de streaming, donde las donaciones grandes son eventos raros pero de alto valor. Una distribución uniforme generaría un promedio sin varianza que no sería interpretable en el dashboard.

4.5. Ajuste horario en eventos de alerta

Los eventos cuya hora generada cae entre las 03:00 y las 12:00 tienen un 70% de probabilidad de ser desplazados 10 horas hacia adelante, concentrando la actividad en el rango de tarde-noche. Esto simula que los streamers transmiten principalmente entre las 18:00 y las 02:00.

Por qué importa: la pregunta de negocio 4 sobre picos de tráfico por día de la semana está relacionada con la distribución horaria. Sin este ajuste, los eventos se distribuirían de forma uniforme a lo largo del día y el indicador de actividad no mostraría patrones reales de uso.

4.6. Fechas coherentes con el ciclo de vida de cada entidad

Las fechas de compra de cada usuario se generan estrictamente después de su fecha de registro, calculando los segundos transcurridos entre el registro y el momento actual con `DATEDIFF(SECOND, ...)` y generando un offset aleatorio dentro de ese rango. El mismo criterio aplica para los eventos de alerta, que se generan estrictamente después de la fecha de creación del proyecto al que pertenece su configuración.

Por qué importa: si las fechas fueran completamente aleatorias sin respetar el orden lógico, podrían generarse compras anteriores al registro del usuario o eventos anteriores a la creación del proyecto, lo que violaría la coherencia semántica de los datos y podría producir resultados incorrectos en las vistas que calculan tendencias temporales.

5. Integridad referencial – orden de inserción

El orden de ejecución respeta las dependencias de claves foráneas entre tablas:

1. **Planes:** sin dependencias externas.
2. **Usuarios:** sin dependencias externas.
3. **EstadosSuscripcion:** sin dependencias externas.

4. **Suscripciones:** depende de Usuarios y Planes.
5. **Proyectos:** depende de Usuarios.
6. **AlertasConfiguraciones:** depende de Proyectos y TipoAlerta.
7. **TipoAlerta:** sin dependencias externas.
8. **CatalogoJuegos:** sin dependencias externas.
9. **JuegosUsuarios:** depende de Usuarios y CatalogoJuegos.
10. **AssetsTienda:** sin dependencias externas.
11. **ComprasAssets:** depende de Usuarios y AssetsTienda.
12. **AlertasEventos:** depende de AlertasConfiguraciones y Usuarios.

6. Validaciones recomendadas después de cargar

Luego de ejecutar los tres scripts en orden, se recomienda correr las siguientes validaciones en SSMS:

Conteo general por tabla:

```
SELECT 'Usuarios' AS Tabla, COUNT(*) AS Total FROM Usuarios
UNION ALL SELECT 'Suscripciones', COUNT(*) FROM Suscripciones
UNION ALL SELECT 'Proyectos', COUNT(*) FROM Proyectos
UNION ALL SELECT 'AlertasConfiguraciones', COUNT(*) FROM
AlertasConfiguraciones
UNION ALL SELECT 'AlertasEventos', COUNT(*) FROM AlertasEventos
UNION ALL SELECT 'ComprasAssets', COUNT(*) FROM ComprasAssets;
-- Esperado: AlertasEventos >= 1500, ComprasAssets >= 1050
```

Verificación de tablas principales:

```
SELECT COUNT(*) AS TotalCompras FROM ComprasAssets;
-- Esperado: 1050
SELECT COUNT(*) AS TotalEventos FROM AlertasEventos;
-- Esperado: 1500
```

Verificación de compradores únicos:

```
SELECT COUNT(DISTINCT IdUsuario) AS CompradoresUnicos FROM
ComprasAssets;
-- Esperado: 24 compradores únicos (40% de 60 usuarios)
```

Verificación de proyectos por usuario:

```
SELECT IdUsuario,
COUNT(*) AS TotalProyectos FROM Proyectos
GROUP BY IdUsuario
ORDER BY TotalProyectos DESC;
-- Esperado: al menos 1 proyecto por usuario, algunos usuarios con
más de 1 proyecto
```

Verificación de distribución de estados de suscripción:

```
SELECT es.DescripcionEstadoSuscripcion, COUNT(*) AS Total
FROM Suscripciones s
```

```
INNER JOIN EstadosSuscripcion es ON s.IdEstadoSuscripcion =
es.IdEstadoSuscripcion
GROUP BY es.DescripcionEstadoSuscripcion;
-- Esperado: presencia de active, trialing, past_due y canceled
```

7. Mapa de distribuciones a preguntas de negocio

Decisión de distribución	Pregunta de negocio que habilita
Distribución de planes 30/30/25/15	P2 — variabilidad de carga por nivel de plan
Upgrades y downgrades forzados por regla	P5 — detección de cambios de plan
Compradores restringidos al 40%	P3 — tasa de conversión de la tienda
Distribución horaria ajustada a tarde-noche	P4 — picos de tráfico por día de semana
Distribución de donaciones en tres niveles	P1 — popularidad y facturación por tipo de asset
Fechas coherentes con ciclo de vida	P6, P7, P8 — análisis temporal mensual y semanal